Group 4. 51%

Digital System Design Coursework – Report 3

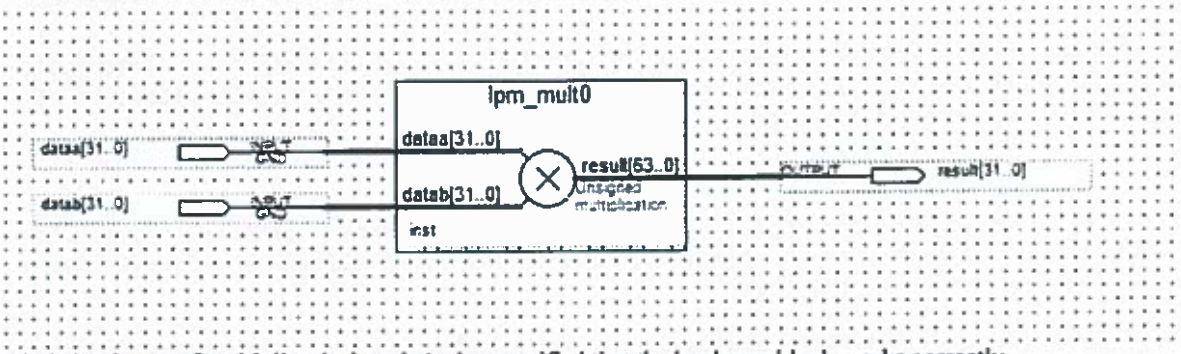
Ben Walker (CID:00837039) and Ju Young Kim (CID:00814764)

Task 6: Add Hardware Floating-Point Units

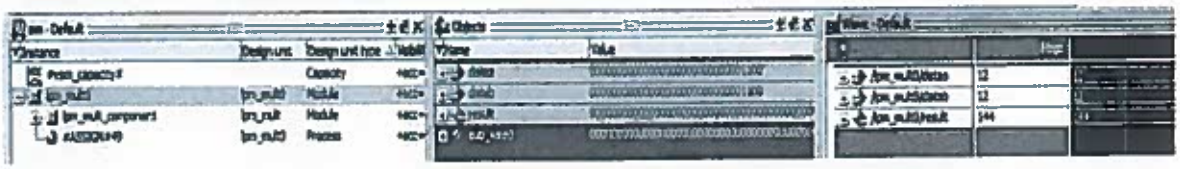
For previous tasks, we did not have a support for floating-point arithmetic in NIOS II. For accurate binary floating-point arithmetic, IEEE-754 floating-point standard was used. In this task, custom hardware blocks that performs floating point adder, subtraction and multiplication, was designed, so that we can use that function to improve the speed of computation for the expense of increased use of FPGA resources.

Add custom instruction tutorial

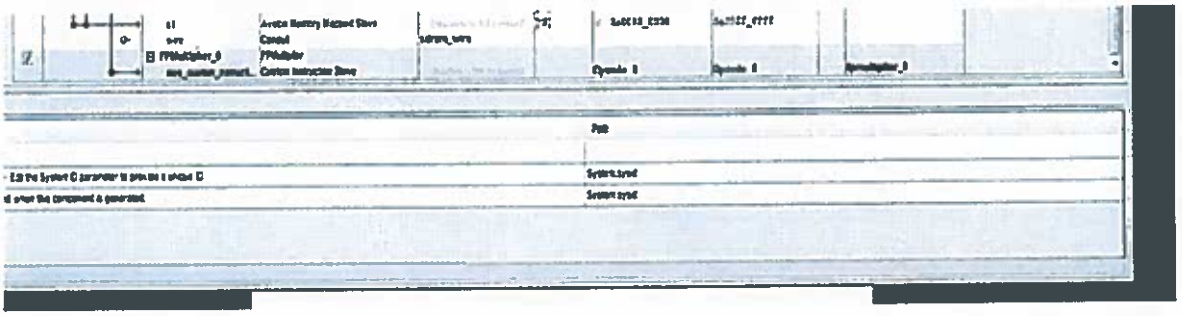
We implemented extended type custom instruction to get a single one-cycle multiplier. The first step was to design a custom hardware block. We selected the LPM_MULT from the Megafunction library, and set two 32 bit inputs and 32 bit output from 32 LSBs from the LPM_MULT component. To verify its functionality, simulation was done through Modelsim in the Quartus environment. Firstly, we generated one HDL block that represents the



whole hardware of multiplier designed. And we verified that the hardware block works correctly.



After generating an HDL design file, we added the hardware block into FPGA using Qsys, so that it can be used in NIOS II system. New NIOS system was generated and the whole deign was compiled.



Assessing the Custom Instruction from Software

Using the NIOS II software build flow, we built a new software project using the Hello World template to run the software that accesses the custom instruction.

```
*/

#define ALT_CI_FPMULTIPLIER_0(A,B) __builtin_custom_inii(ALT_CI_FPMULTIPLIER_0_N,(A),(B))
#define ALT_CI_FPMULTIPLIER_0_N 0x0
```

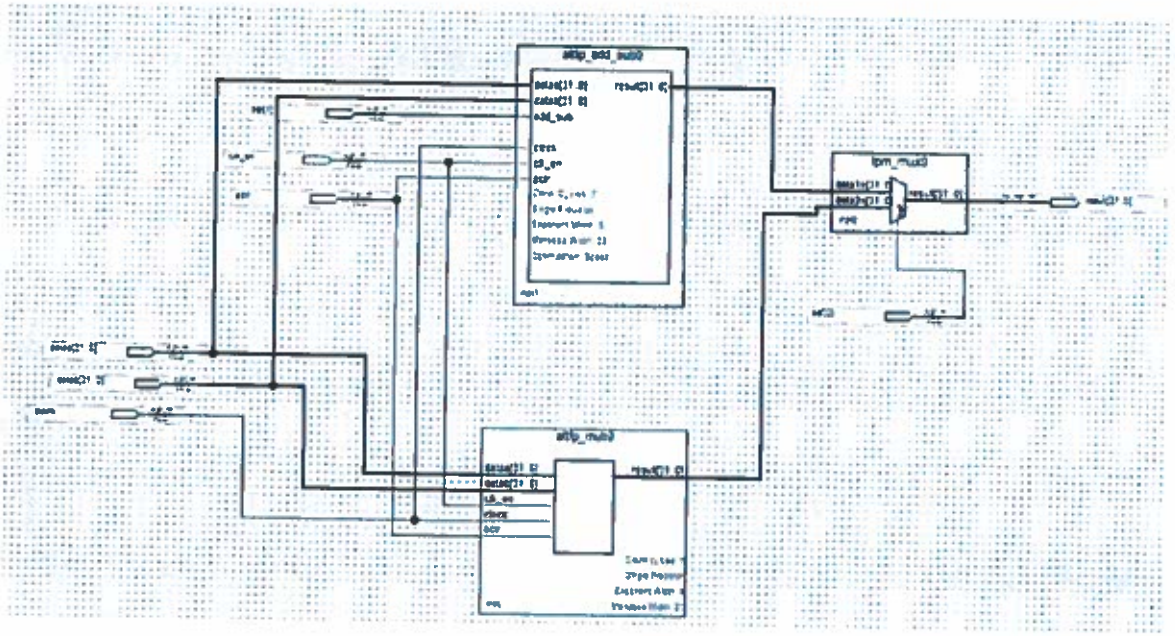
To verify its functionality, we included the system.h file to the code of hello_world.c files, so that we can access the custom instruction. The software produced 2 results, one from the original multiplication function and one from the custom hardware that was added. They gave out same answer, proving that the custom instruction was properly run.

```
Hello from Nios II!
Multiplication result: 8
Multiplication result from custom instr: 8
```

After getting us familiarized with the process of adding custom instructions, custom instruction for floating point arithmetic was added to our system, because the target function involves use of floating-point arithmetic heavily and we want to accelerate the computation.

Firstly, we implemented the custom logic blocks for floating-point adder, subtracter and multiplier operations by floating-point addition/subtraction and multiplication hardware block from Megafunctions, and combined them using a 2-to-1 multiplexer with select input.

We chose type of custom instruction as Multicycle, because we needed clk, clk_en and reset ports and we can specify number of clock cycles to complete an operation. For each of floating-point custom software, we chose the smallest output latency that we can have, because we found out that smaller the output latency, the less FPGA resources is used and the faster the speed of operation becomes.



We compiled the whole hardware design and got resources used.

Resource that occupy on the FPGA for float_mult block.

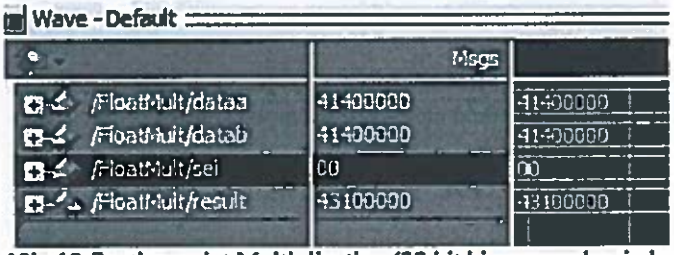
Resources	Values
Logic Elements	1,067/15,408 (7%)
Registers	551
Memory Bits	45/516,096 (< 1%)
Embedded Multipliers	7/112 (6%)
PLLs	0/4 (0%)
FPGA Resource	0.043,946

Having additional more cycles than each of hardware blocks' output latency was important, so that any sub-hardware block performs correctly with enough clock cycle for one operation. "altfp_mult0" block has output latency of 5 and "altfp_add_sub0" has output latency of 7. Thus, latency of float_mult was chosen as 8 cycles.

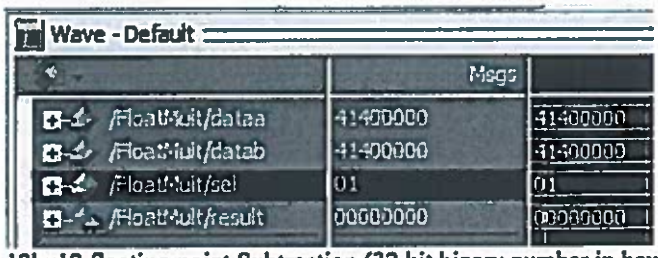
Truth table for overall design depending on select input, sel[1:0]

Select	Selected Block	Arithmetic Output
X0	altfp_mult0	dataa x datab
01	altfp_add_sub0	dataa - datab
11	altfp_add_sub0	dataa + datab

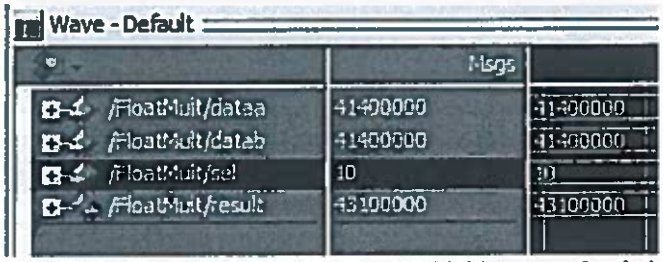
This new hardware block was run by MODELSIM to verify its functionality.



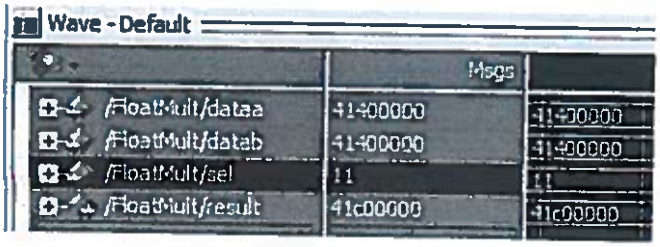
12by12 floating point Multiplication (32 bit binary number in hexadecimal)



12by12 floating point Subtraction (32 bit binary number in hexadecimal)



12by12 floating point Multiplication (32 bit binary number in hexadecimal)



- 12by12 floating point Addition (32 bit binary number in hexadecimal)

We generated HDL file of whole design, and added the hardware block to FPGA using Qsys.



Using custom instructions, we run the software in eclipse to get the results for all 3 cases. We compared the results for the target function with task 5 for all 3 cases. Task 5 results with embedded multipliers were chose rather than the result from using logic elements, because embedded multiplier was faster than logic element with a slight more use of resources.

- Execution time(latency) that computes F(x)

Test case	Task 5 (Embedded Multipliers)	Task 6
Test case 1	59	55
Test case 2	2,723	2,580
Test case 3	278,173	255,397

- Comparing with task 4, the computational speed increased more than twice with around 0.01~0.02 of more FPGA resources used in task 5, with support of multiplier support such as logic elements and embedded multipliers.

-In task 6, we implemented custom instruction to put all the operators to work in hardware rather than being run in software. It did increased the speed, but not significantly, because the cos function is still computed with multiplier support.

- Resource utilization(Cache size of 8Kbytes)

Resources	Task 5 (Embedded Multipliers)	Task 6
Logic Elements	3,407/15,408 (22%)	5,080/15,408 (33%)
Registers	2,258	2,926
Memory Bits	163,840/516,096 (32%)	163,885/516,096 (32%)
Embedded Multipliers	4/112 (4%)	7/112 (6%)
PLLs	1/4 (25%)	1/4 (25%)
FPGA resource	0.193333	0.236667

-With cost of improvement in performance, it increased speed of operation.

Code size of NIOS II applications for the different vector sizes

Test case	Text (T5)	Text (T6)	Data (T5)	Data (T6)	Bss (T5)	Bss (T6)
Test case 1	47,636	66,522	1,780	6,748	336	748
Test case 2	47,144	66,433	1,780	6,748	336	748
Test case 3	48,644	67,500	1,780	6,748	336	748

- Error btw the result of our system and the reference double precision Matlab implementation for each

A-

test case

Test case	Program Answer (T5)	Program Answer (T6)	MATLAB Answer	Relative Error (T5)	Relative Error (T6)
Test case 1	-57,879	-57,879	57879.87297	0.00150825	0.00150825
Test case 2	-76,972	-76,973	-76972.17496	0.000227307	0.00107185
Test case 3	37,022,697	37,022,500	37,022,698.00514260	0.000002714936116	0.0005348238

Relative error increased by factor of 5 for test case 2 and by factor of 250 for test case 3.

Task 7: Add Dedicated Hardware Block to compute the inner part of the arithmetic expression

In this task, we implemented the whole arithmetic function as a custom instruction in our system. Custom instruction allows us to implement all the operations as a hardware block rather than being run as a software, increasing computation speed.

$$y = \sum_{i=1}^{N} 0.5x_i + x_i^2 \cos\left(\left\lfloor \frac{x}{4} \right\rfloor - 32\right)$$

Firstly, we implemented cos(.) through implementation of a CORDIC block, and the argument is given in integer form. CORDIC block allows us to compute complex functions efficiently without much resource cost, like including multiplier.

CORDIC algorithm is based on rotating unit-length vector by an angle z, so new vector will be placed at (cos z, sin z). CORDIC rotation involves 3 variables, which are x, y and z

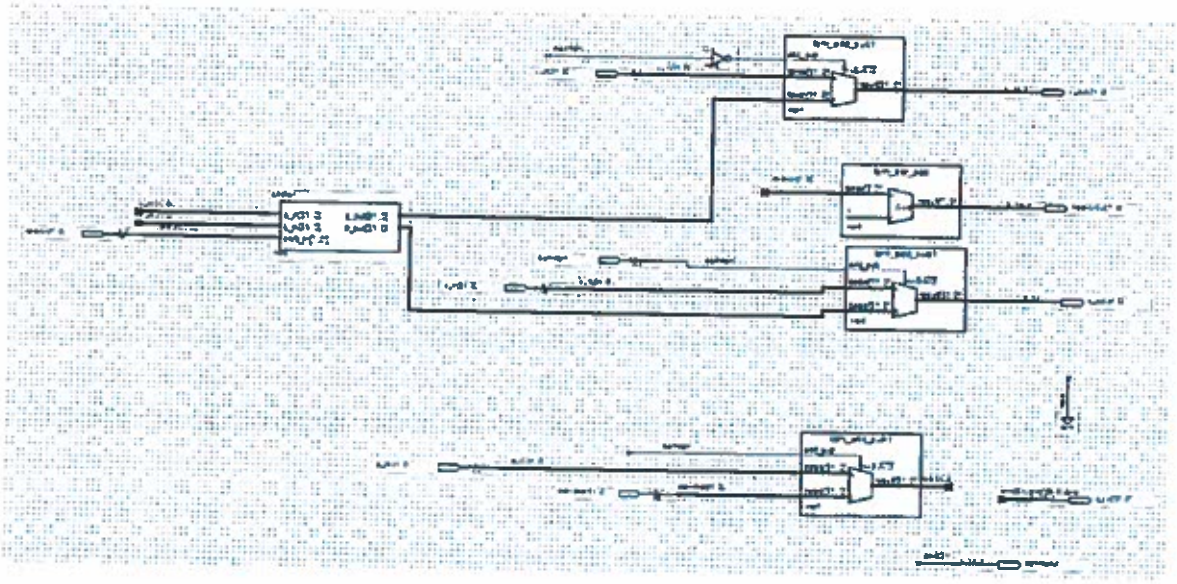
$$x_{i+1} = x_i - y_i tan(\alpha_i) = x_i - d_i y_i 2^{-i}$$

$$y_{i+1} = y_i + x_i tan(\alpha_i) = y_i + d_i x_i 2^{-i}$$

$$z_{i+1} = z_i - \alpha_i = z_i - d_i \alpha_i$$

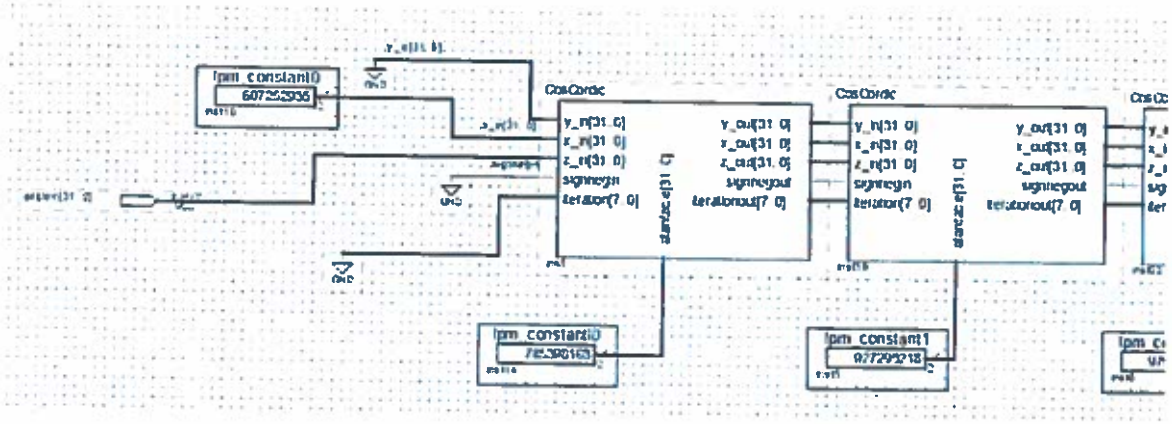
$$d_i \in \{-1,1\}$$

Thus, we can get values for x, y and z by addition and shift, not using any multiplier. We implemented x, y, z and i by using lpm addition/subtraction modules and shifter.



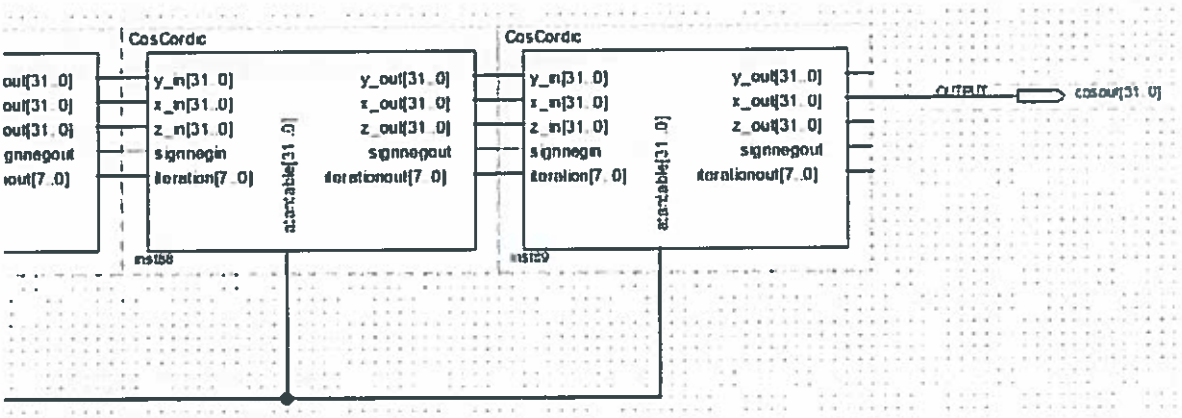


We have 32 stages, which is corresponding to the number of bits we use. Number of stages directly relates to the level of precision of rotations and accuracy of the CORDIC results



Full CORDIC circuit has input ports of initial values of x, y, z, sign and iteration number i. As it passes through the stages, values of x, y, z, sign and iteration number I changes, output value gets more and more accurate.

I would expect to see an analysis on the # bits/iteration for CORDIC.



This subsequent 32 CORDIC modules results in accurate cosine value of a given constant. By having CORDIC circuit, there is a huge improvement in computation speed, because cos function was the main heavy calculation that took much of an execution time.

We Compared performance of task 7 with CORDIC circuit, with task 6 for all 3 cases.

- Execution time, latency that computes F(x)

Test case	Task 6	Task 7
Test case 1	55	
Test case 2	2,580	
Test case 3	255,397	

Because we mapped all cosine values to hardware by using CORDIC algorithms, the speed of computation of the target function got much faster

- Resource utilization

Resources	Task 6	Task 7
Logic Elements	5,080/15,408 (33%)	8,529/15,408 (55%)
Registers	2,926	2,321
Memory Bits	163,885/516,096 (32%)	163,840/516,096 (32%)
Embedded Multipliers	7/112 (6%)	4/112 (4%)
PLLs	1/4 (25%)	¼ (25%)
FPGA resource	0.236667	0.3033

As a cost, there was increased use of resources.

- Code size of NIOS II applications for the different vector sizes

Test case	Text (T6)	Text (T7)	Data (T6)	Data (T7)	Bss (T6)	Bss (T7)
Test case 1	66,522		6,748		748	
Test case 2	66,433		6,748		748	
Test case 3	67,500		6,748		748	

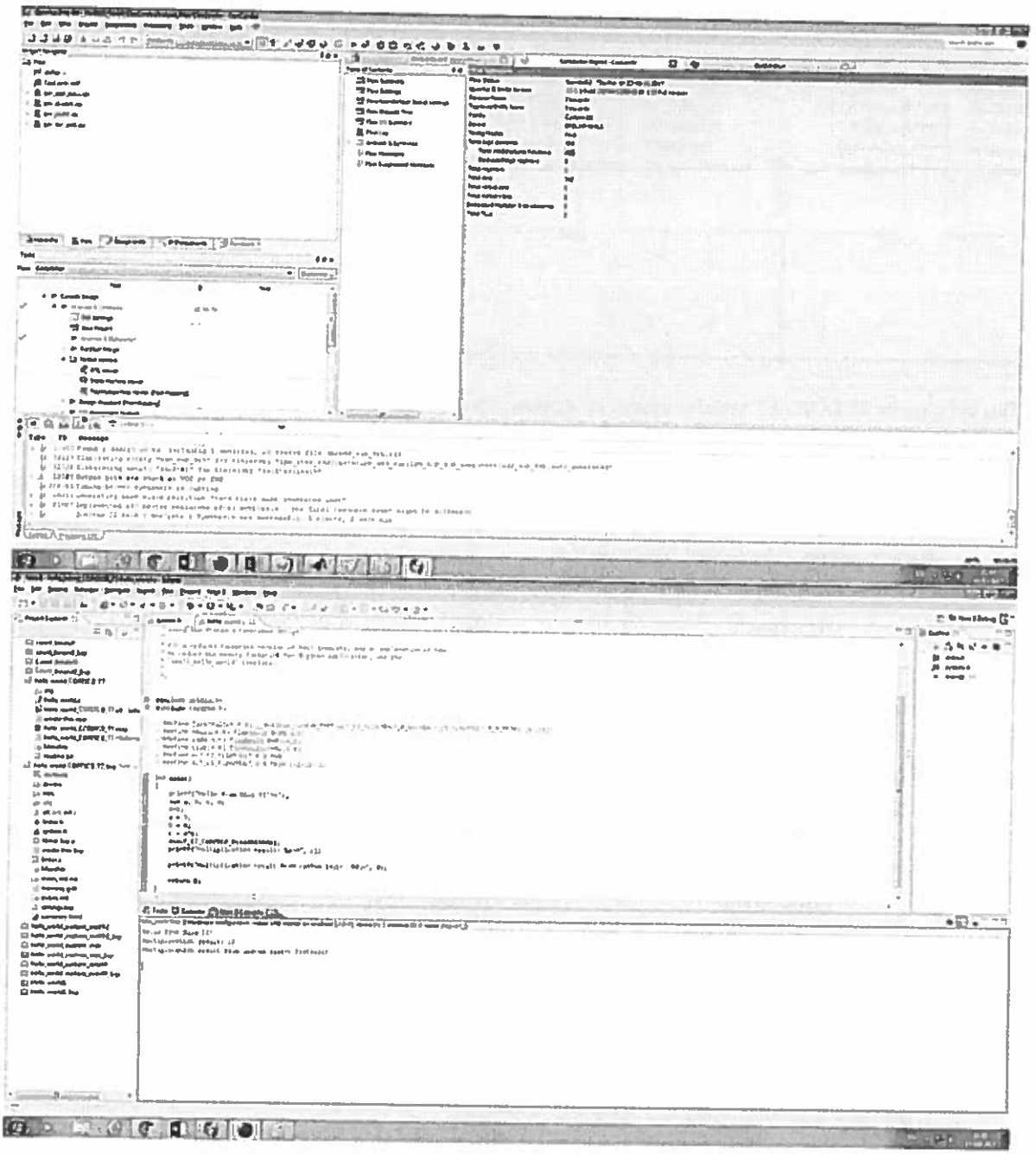
- Error btw the result of our system and the reference double precision Matlab implementation for each test case

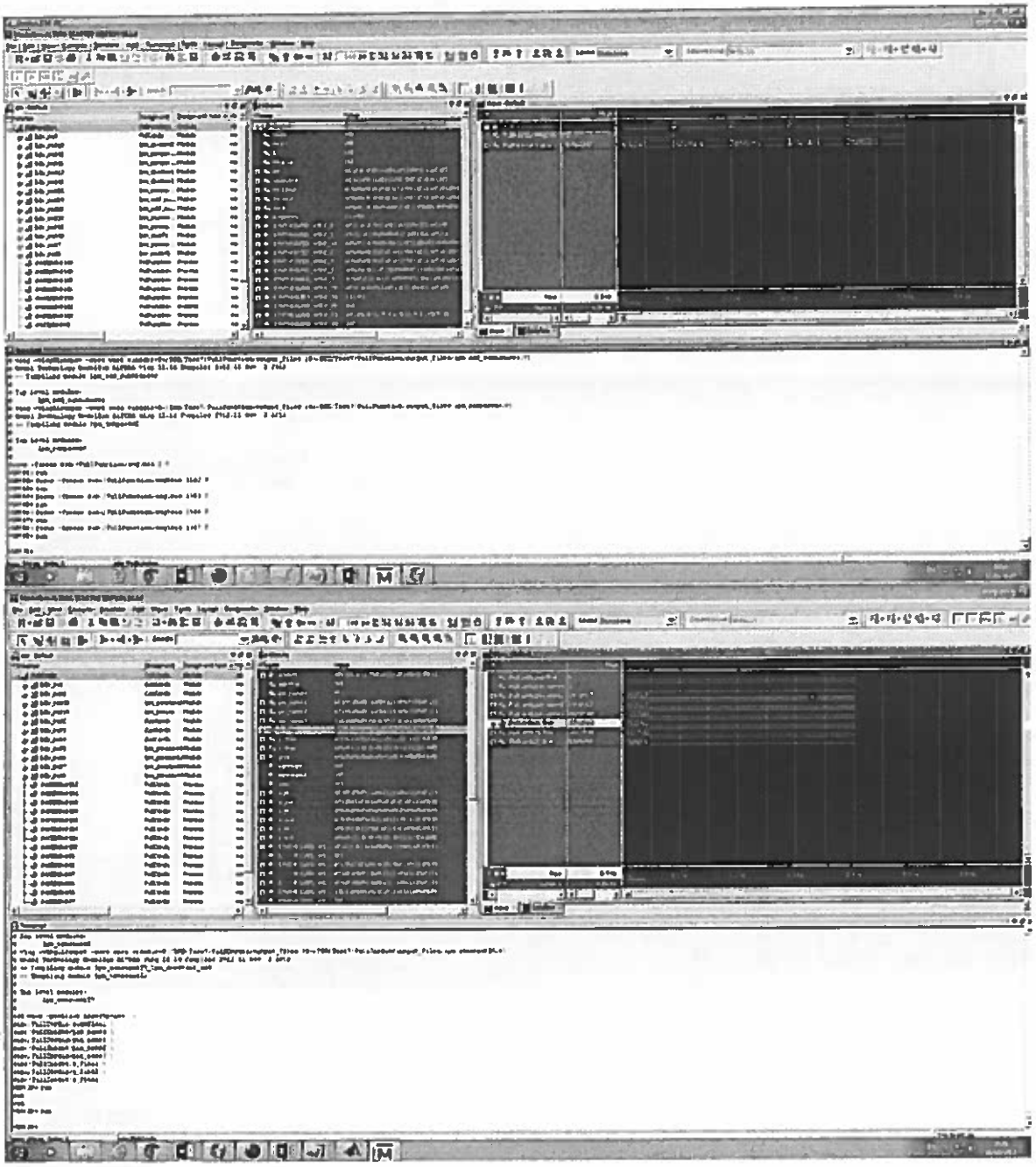
Test case	Program Answer (T6)	Program Answer (T7)	MATLAB Answer	Relative Error (T6)	Relative Error (T7)
Test case 1	-57,879		57879.87297	0.00150825	
Test case 2	-76,973		-76972.17496	0.00107185	
Test case 3	37,022,500		37,022,698.00514260	0.0005348238	

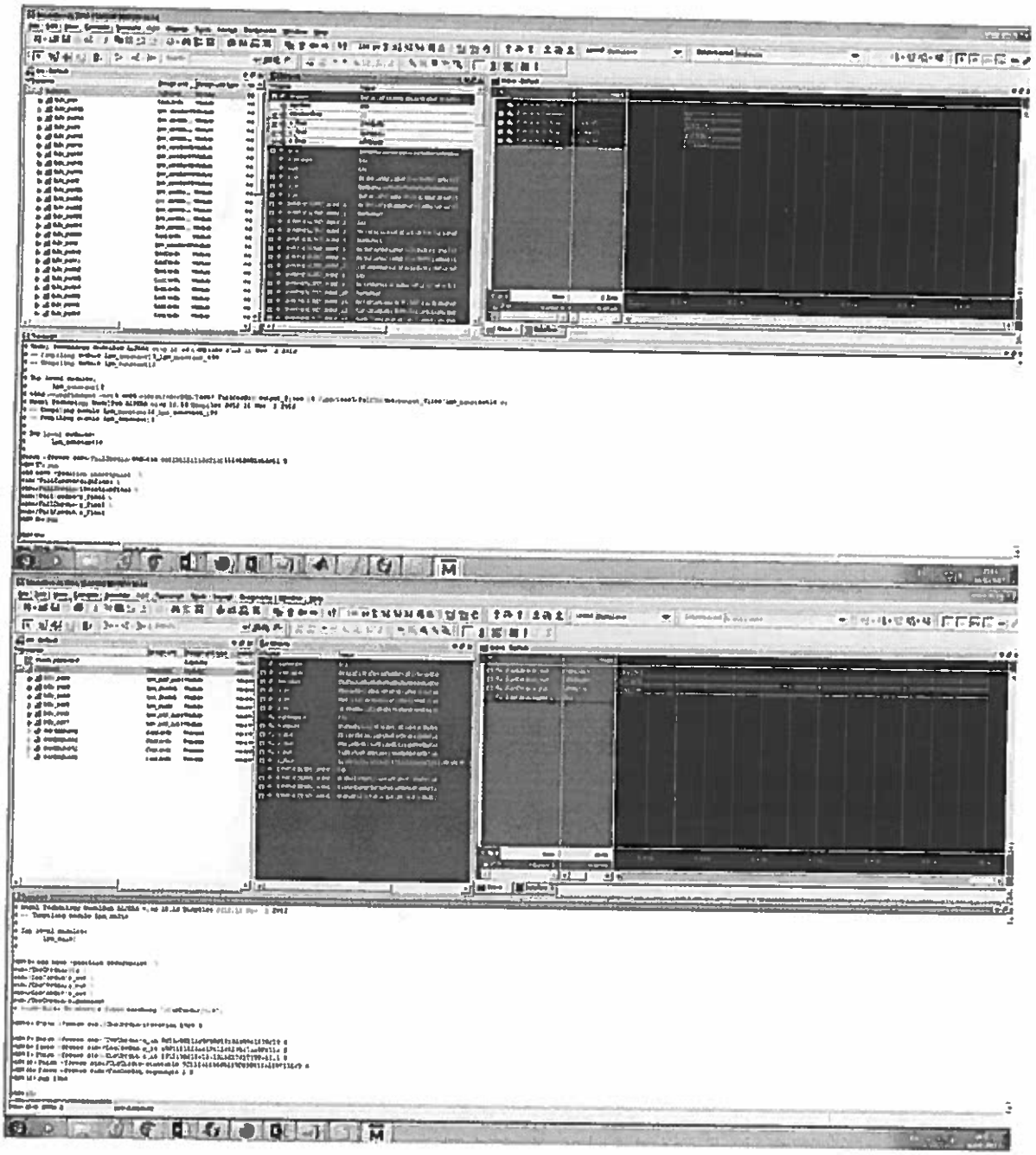
C+

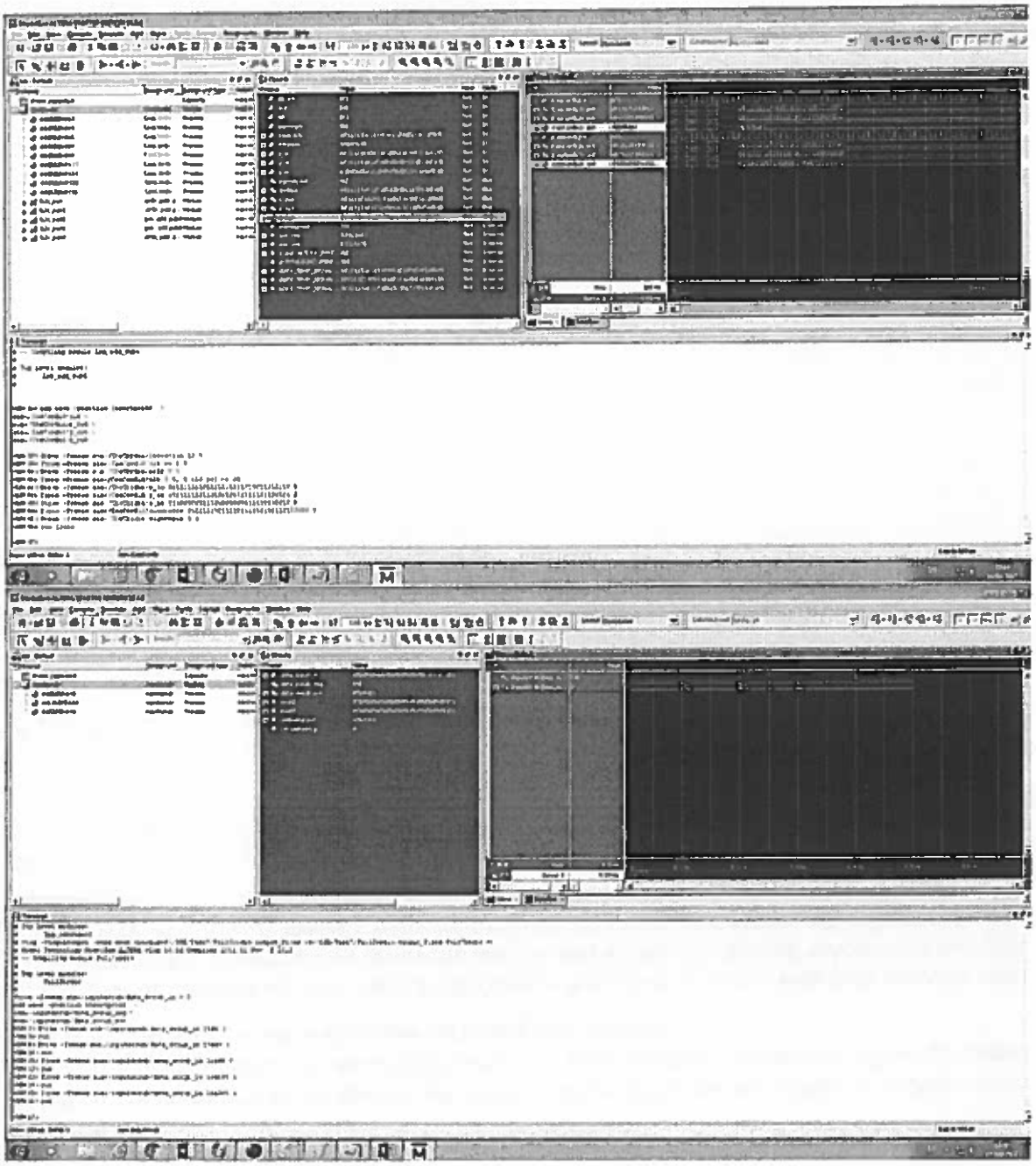
C

The design of CORDIC is clean, but

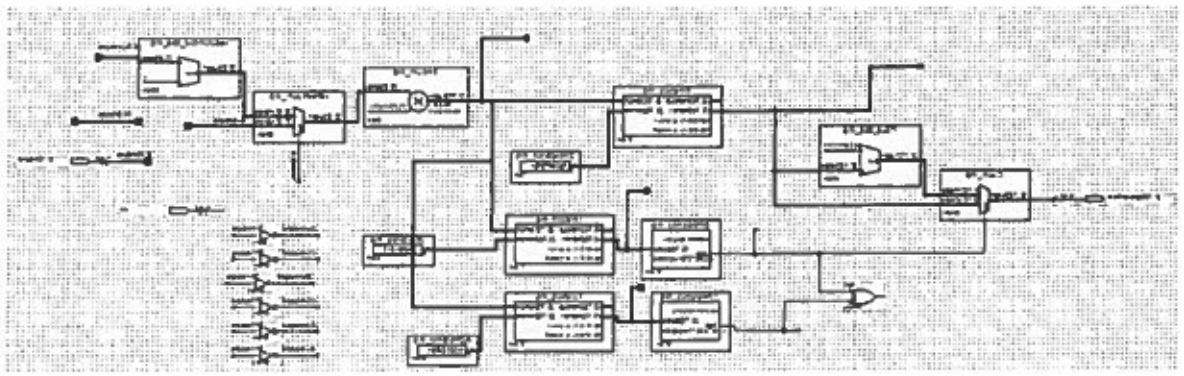








Full functions that include the full 32 CORDIC functions



Task 8: Add Dedicated Hardware Block to compute the arithmetic expression



Conclusion

In Task 6, we learnt how to use custom instructions to make all the operators work in hardware rather than being run in software. Because computation of the target function involves a lot of floating number calculation, we added a custom instruction for floating point arithmetic.

In Task 7, we implemented cos values using CORDIC algorithms, which allowed us even faster operation speed. CORDIC algorithms allowed us to compute cos values with only adder and shifter, and without heavy multiplier. With 32 subsequent CORDIC modules, we implemented it as custom instruction and run with software to compute the target function.

We were able to run the algorithm using CORDIC (although not the full implementation), which gave greatly increased speeds. Although the full algorithm wasn't quite finished, we managed to execute task 3 in 44467 ticks.

but.
you said that
you could not run
Task 7.

Appendices

Code for all tasks